Burning Tears of PHP's Memory Hardening

A fully remote, generic exploit path that survives PHP's newest heap defenses, from one constrained byte.

Yifan Wu · Xiaochuan Yu · Zhiyun Qian · UC Riverside · UC San Diego

This research is 100% LLM-free.

(except for making these slides)

Two researchers, one lab.

NEBULA SECURITY

black hat

nebula@blackhat: ~/talks/burning-tears

```
nebula@blackhat:~$ whoami
```

Frank Wu · Nebula Security
Hacking Linux and Android

Xiaochuan Yu · Nebula Security
Hacking XNU and Browsers

```
nebula@blackhat:~$ cat recent_work.md
```

- first public nginx RCE · remote, ASLR bypass, generic config
- world's first Android 17 root (GhostLock) · one URL, full device control
- first public Android browser → kernel full-chain (IonStack) in 7 years

```
nebula@blackhat:~$ ./burning_tears.sh --target=php --remote
```

It started with a CTF.

r3kapig

✅ securinets-ctf-quals-2024 › ✅ Pwn-I HATE PHP!

Frank Wu ⚡NEBU PWN 2024/10/13 03:10

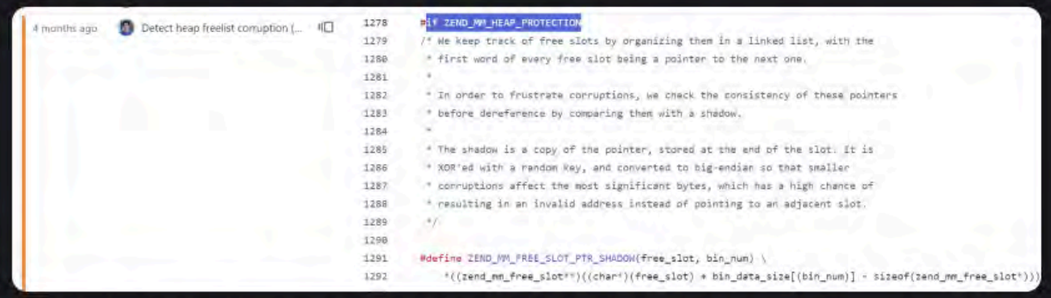
maybe we are close to aaw..

```
*RDI  0x76838d200040 ← 0
*RSI  0x76838d201060 → 0x76838d201090 → 0x76838d2010f0 → 0x76838d1f3e90 → 0x63f0e5fc25 (_efree) ← ...
R8   0
```

👍 1

Frank Wu ⚡NEBU PWN 2024/10/13 03:41

WT



PROTECTION added right before 4 month

Thank you Securinets CTF 🥰🥰

1. **The CTF**
   An ordinary PHP challenge.
2. **The 0-day**
   A real flaw. PHP patched it.
   AN UNPLANNED SIDE-FIND
3. **The real question**
   One defense cracks, all four?

One request. That's all you get.

Browser pwn

```
for(let i=0;i<0x4000;i++){
    foo(true);
}
new ArrayBuffer(0x7f00000);
```

Kernel pwn

```
for(int i=0;i<N;i++)
  msgsnd(qid[i],&msg,size,0);
```

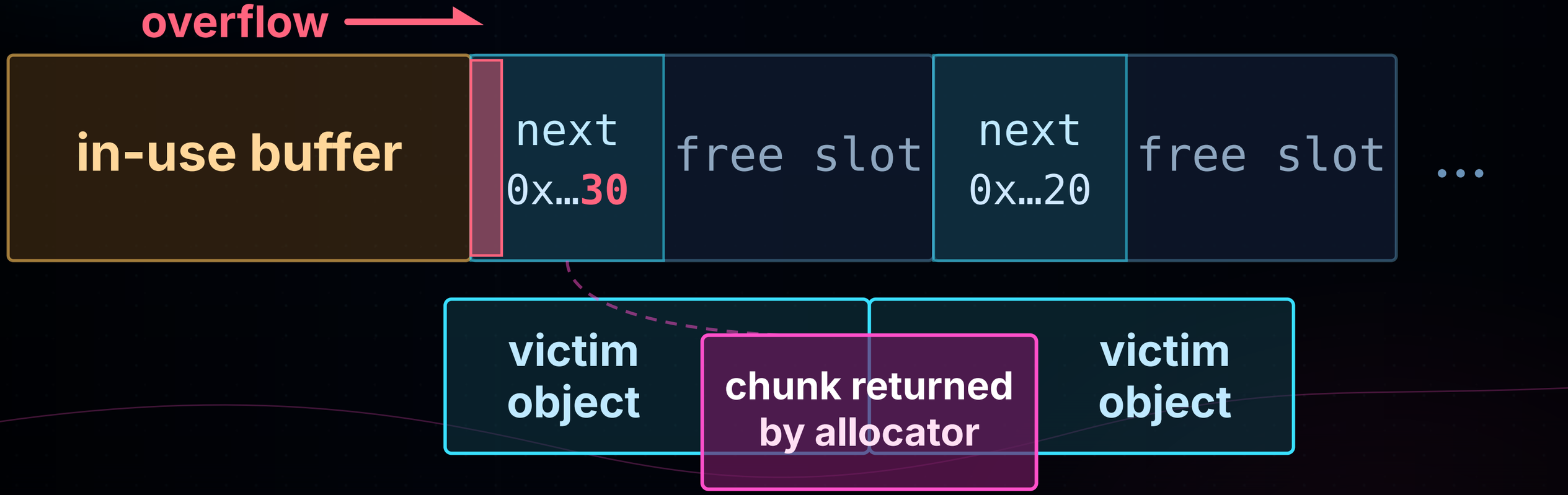
PHP remote

```
POST /upload.php HTTP/1.1
Host: victim.tld
Content-Length: 8192

data=%00%02%be%ef...
```

How you used to pwn PHP.

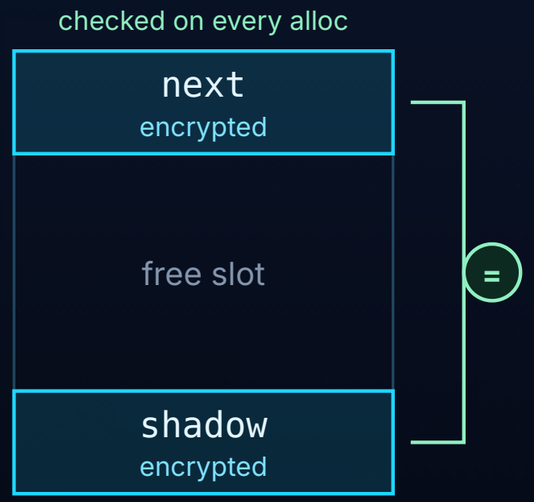
ZendMM: singly-linked list



read / write victim fields › leak & overwrite a function pointer › RCE

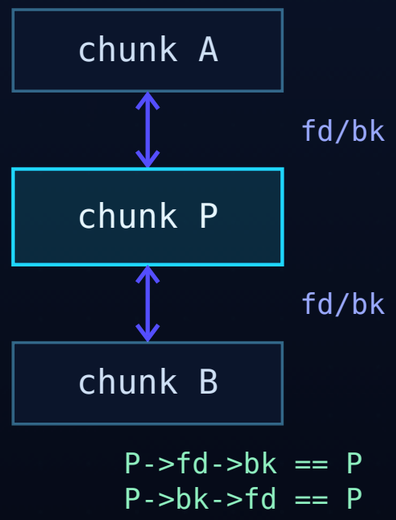
Then PHP fought back.

Shadow Pointer



× freelist poisoning

Unlink Prevention

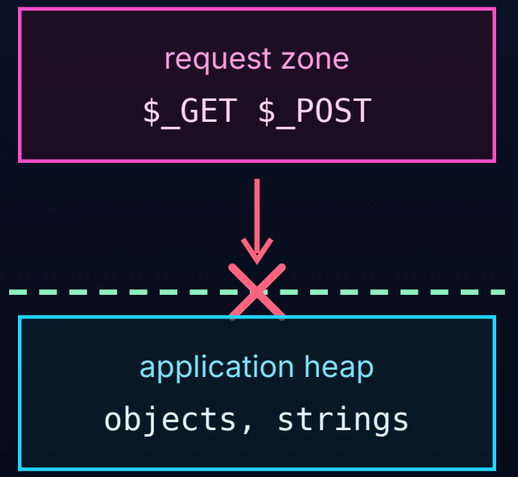


× list-forgery writes

RO Metadata

× metadata hook hijack

Heap Isolation



× raw-request spray

~~CVE-2024-2961~~ ~~CVE-2022-31626~~ ~~public CTF chain~~

Aimed at the exact techniques every public PHP exploit relied on.

The old exploits died.

~~freelist next~~

blocked by Shadow Pointer

~~metadata hooks~~

blocked by Read-only

~~$_POST spray~~

blocked by Heap Isolation

Each new defense closes exactly one classic technique
freelist, metadata, and raw spray all gone.

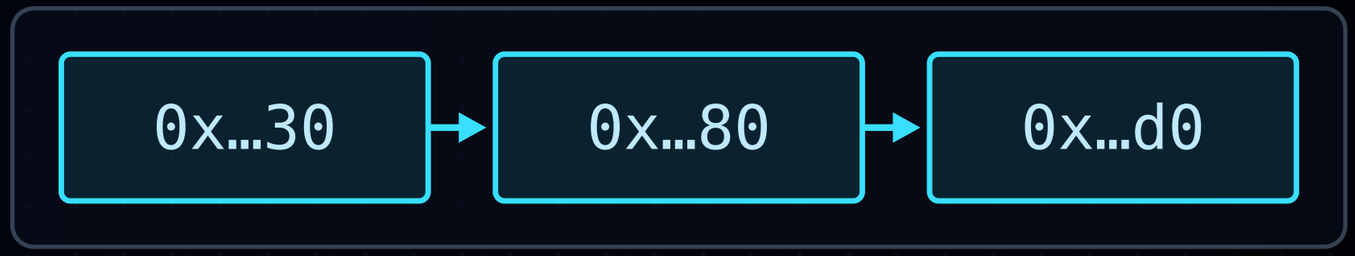
PART I
The playing field

How PHP allocates, and why that lets us shape the heap at all, remotely.

Same request, same heap.

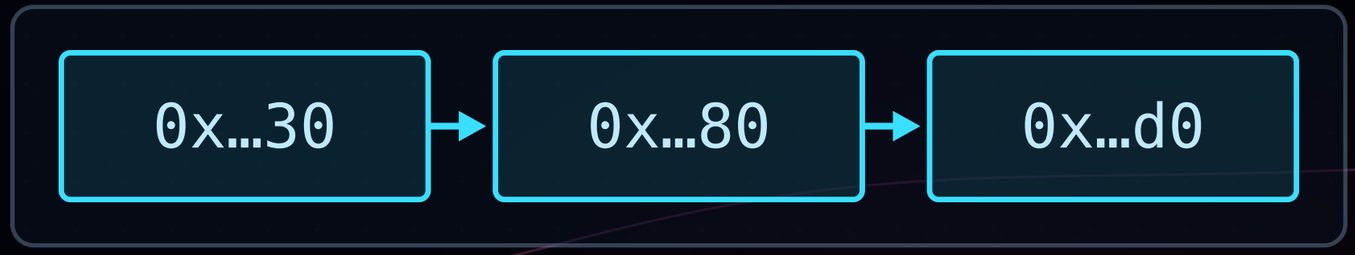
request #1 freed pages

freelist



request #2 freed pages

freelist



Fresh chunk each request, best-fit + LIFO, no cross-request state,
so identical requests build identical heaps.

What fengshui buys us.

Anything (raw bytes)

Spray attacker-defined raw memory the interpreter will later trust.

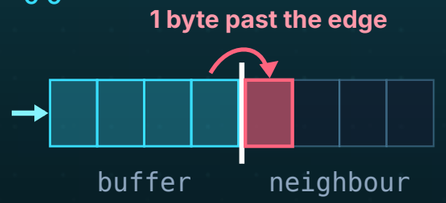
Anywhere (placement)

Put a chosen object at any offset next to the victim, in one request.

Both are hard under the new defenses.
Earning them back is the rest of the talk.

So we stopped touching the allocator.

00



1-byte OOB

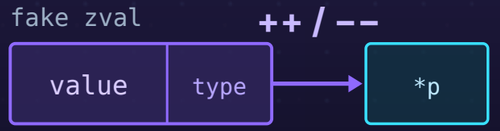
the weakest possible bug

01 CONTRIBUTION

Index Forgery

corrupt a table index, not a pointer

02



Arbitrary ++/--

read = *p++, write = *p--

03

- leak
- probe
- hijack

ZOP

the interpreter is the gadget set

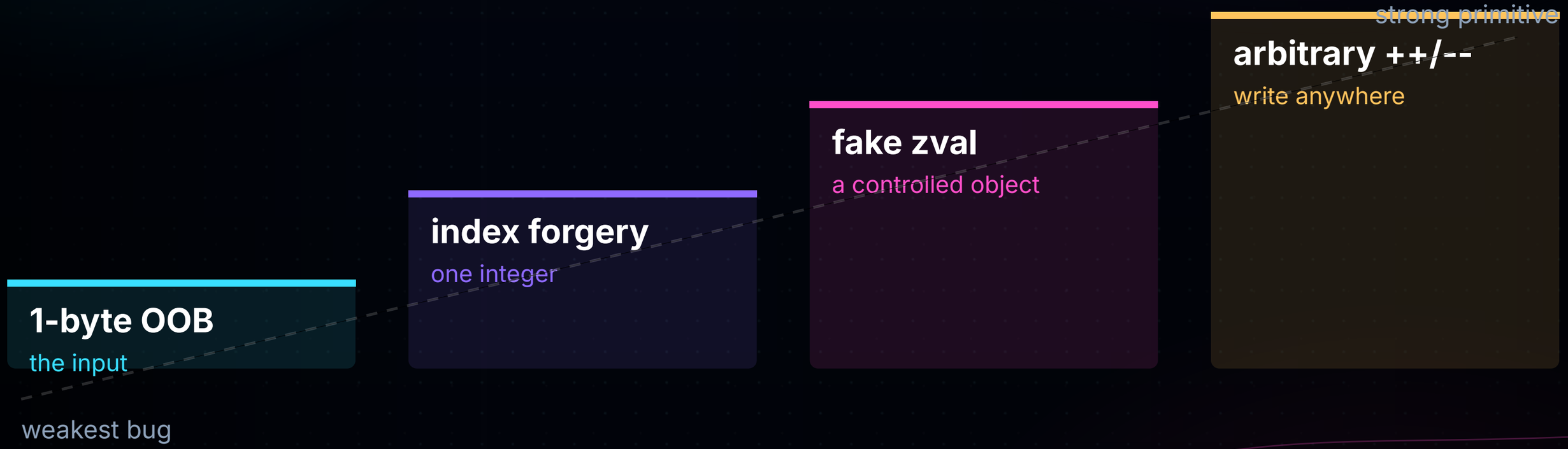
Every step lives in built-in PHP objects, never the allocator, never app classes.

II

Index Forgery

Turning the weakest possible bug, one constrained byte, into a strong primitive.

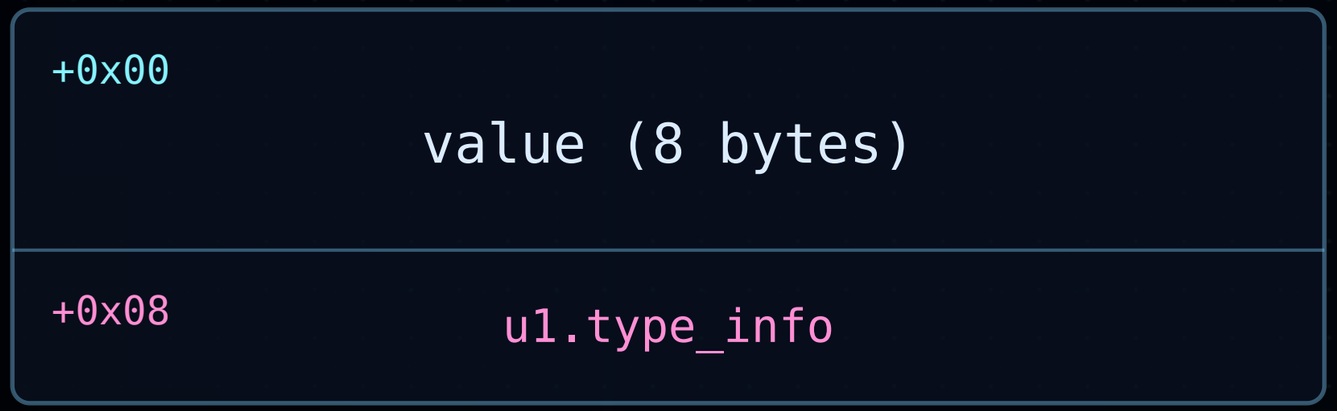
One byte in.



1-byte overflow | value constrained | no OOB read

The nightmare bug, yet enough. Works here means it works for almost anything stronger.

Meet the zval.



flip the low type byte -> same 8 bytes reinterpreted

IS_LONG	= 4
IS_DOUBLE	= 5
IS_STRING	= 6
IS_ARRAY	= 7
IS_OBJECT	= 8

flags byte: IS_TYPE_REFCOUNTED = 1<<0

Every PHP value is 8 bytes plus a type tag. Control a zval, and you control what PHP believes memory is.

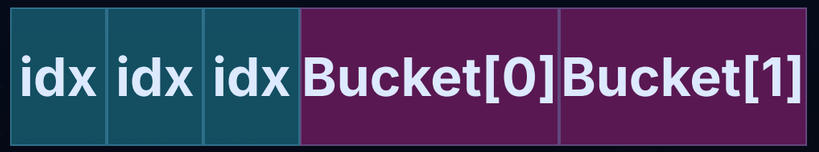
The real target: arData.

ZEND_ARRAY (HASHTABLE)

+0x00	**gc** · refcount
+0x0c	**nTableMask**
+0x10	**arData** → butterfly
+0x18	**nNumUsed** · size
+0x38	**pDestructor** · hijack target

ARDATA IS A BUTTERFLY

A *separate* allocation the overflow reaches. arData points to the split: index array on the left half, buckets on the right.



← index grows | arData → buckets grow →

A normal lookup.

A zend_array reads from a separate arData buffer, index array in front of the buckets.

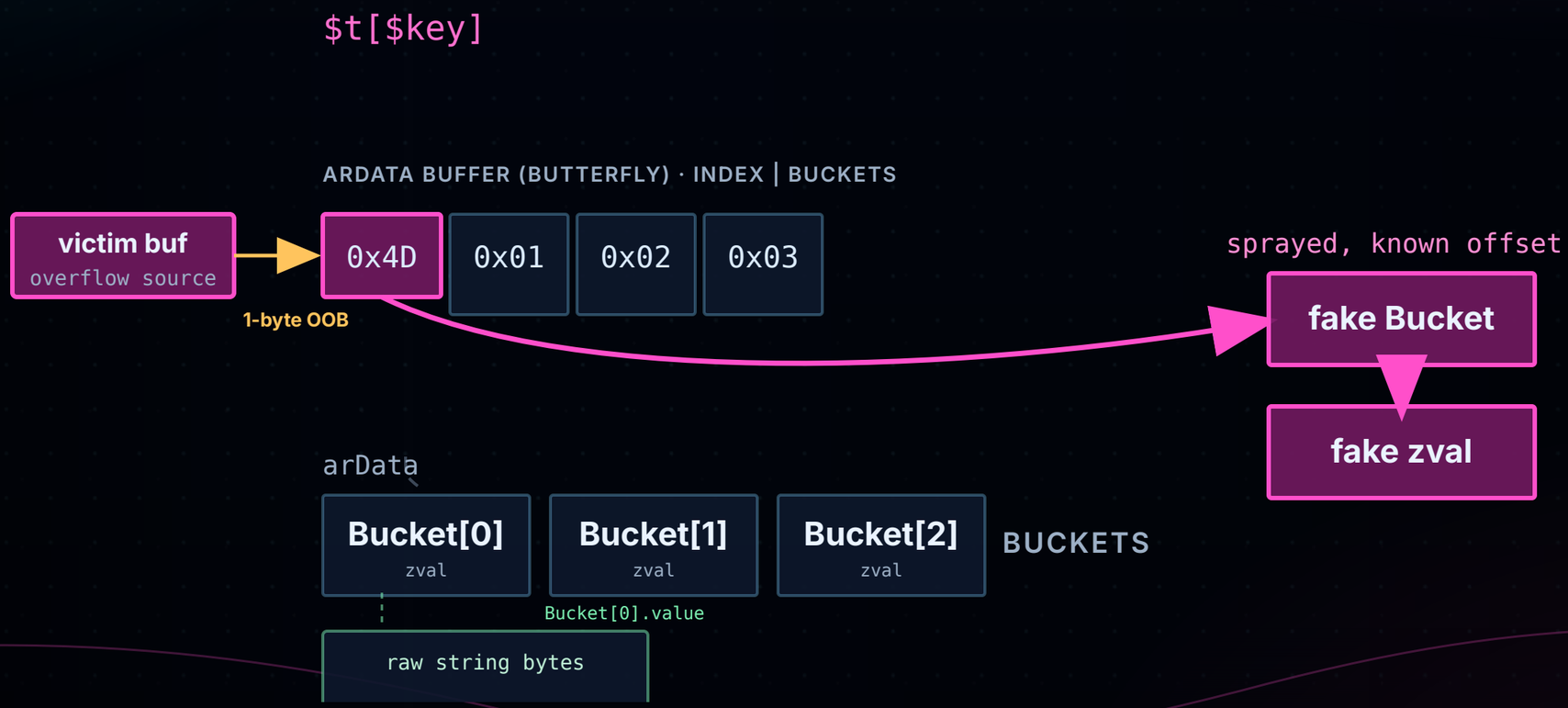
A normal lookup.

hash(key) | mask → a slot → the integer there picks the Bucket. PHP never doubts that integer.

One byte, one redirect.

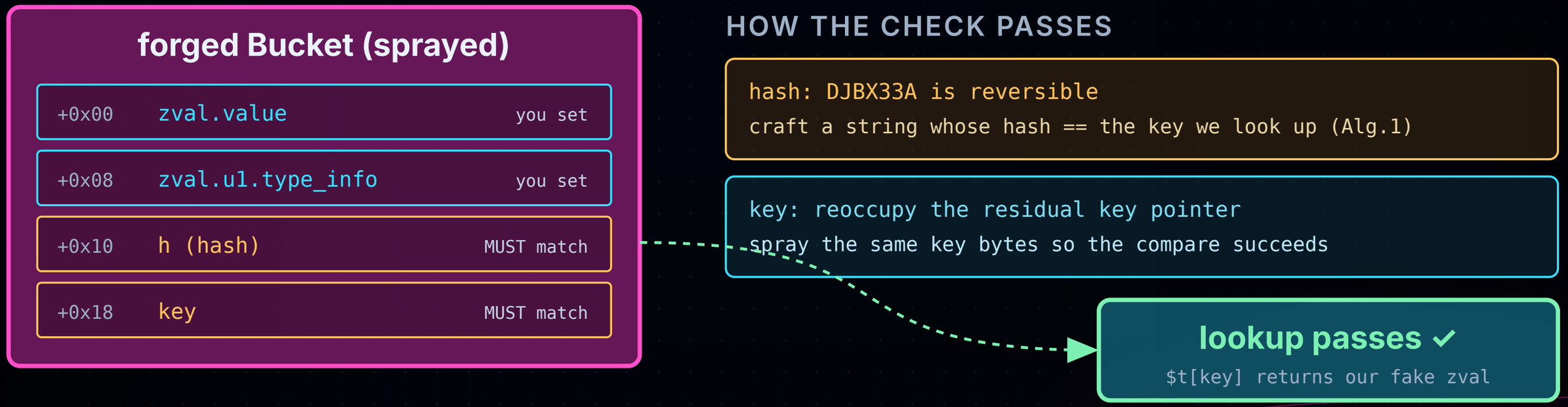
The overflow lands onindex[0], not a pointer, just a small integer. One byte moves it far.

One byte, one redirect.



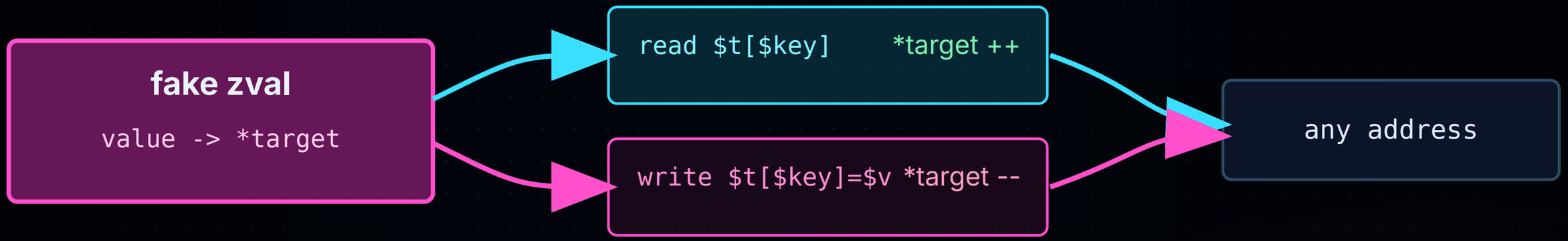
A large index reaches memorywe sprayed, a fake Bucket holding a fake zval we control.

Make the fake pass.



PHP re-checks the bucket key and hash. We forge a match with the reversible DJBX33A hash, and the lookup passes.

Now you own a zval.



Mark the fake zval refcounted, and a plain read/write does ++ / -- through your pointer, at any address.

But the pointer is constrained.

WHY LIMITED

The value pointer is sprayed as a string, encoding-restricted. We control only its low bytes.

So ++/-- lands *near* a chosen spot, not anywhere.

value = 0x...??

low-byte overlap

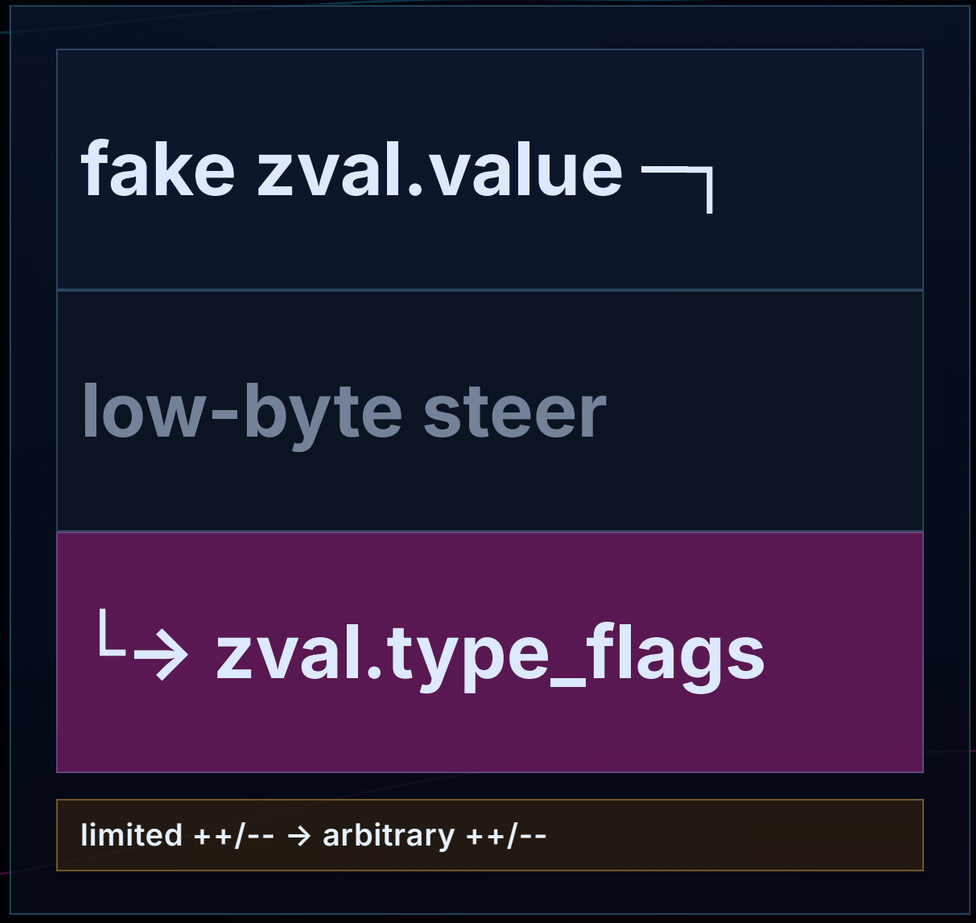
→ nearby sprayed memory

Lift to arbitrary.

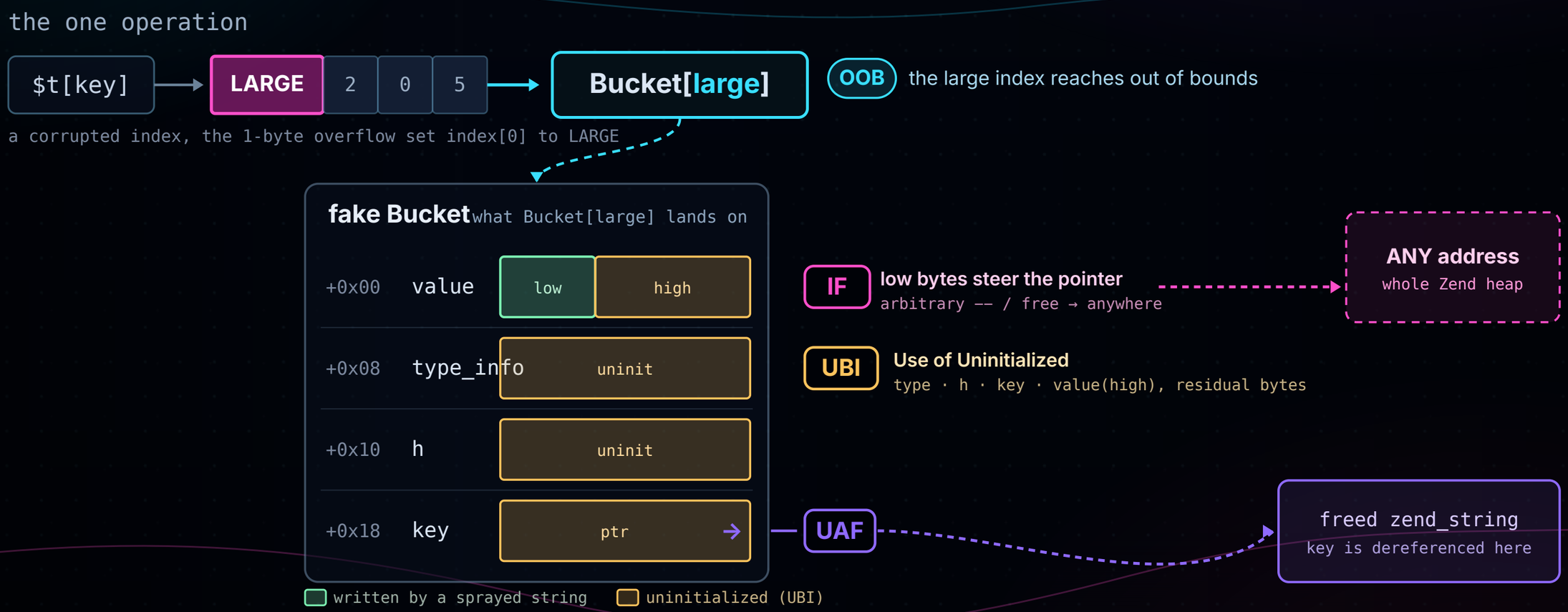
ONE EXTRA INDIRECTION

Steer the constrained value at the type_flags of another zval in a sprayed array.

++/-- on a type tag → make any zval refcounted → decrement anywhere.



One corrupted index. Four primitives.



The **out-of-bounds** access lands on a fake Bucket that is mostly **uninitialized (UBI)**, only the low bytes of `value` are string-written; its `key` is dereferenced into a freed string (**UAF**), and steering that value pointer gives **IF**, an arbitrary decrement anywhere.

OOB and UAF meet here.

Different bugs, one waypoint. OOB via Index Forgery and UAF both reach a controlled zval, then run the same ZOP.

ZOP

Zend-Oriented Programming, the interpreter's own structures become the gadget set.

The interpreter is the gadget set.

FROM ONE PRIMITIVE, EVERYTHING

arbitrary ++/-- → leak · probe · hijack

LEAK

flip a type tag, PHP prints the pointer

PROBE

++/-- as an oracle to defeat ASLR

HIJACK

a destructor pointer you control

Let PHP print the pointer.

returned in JSON/XML -> Zend heap address leaked

No OOB read. Flip a string zval to a double and the same bytes come back as a number in the response.

ASLR → just probing.

PHP SIDE (NEED PHP-FPM / ZIF_SYSTEM)

php-fpm .text	0x000059adca400000	r-x

near each other

FPM_heap	0x000059adf7410000	rw-

LIBC SIDE (NEED SYSTEM)

libc.so.6	0x00007ffff780d000	r-x

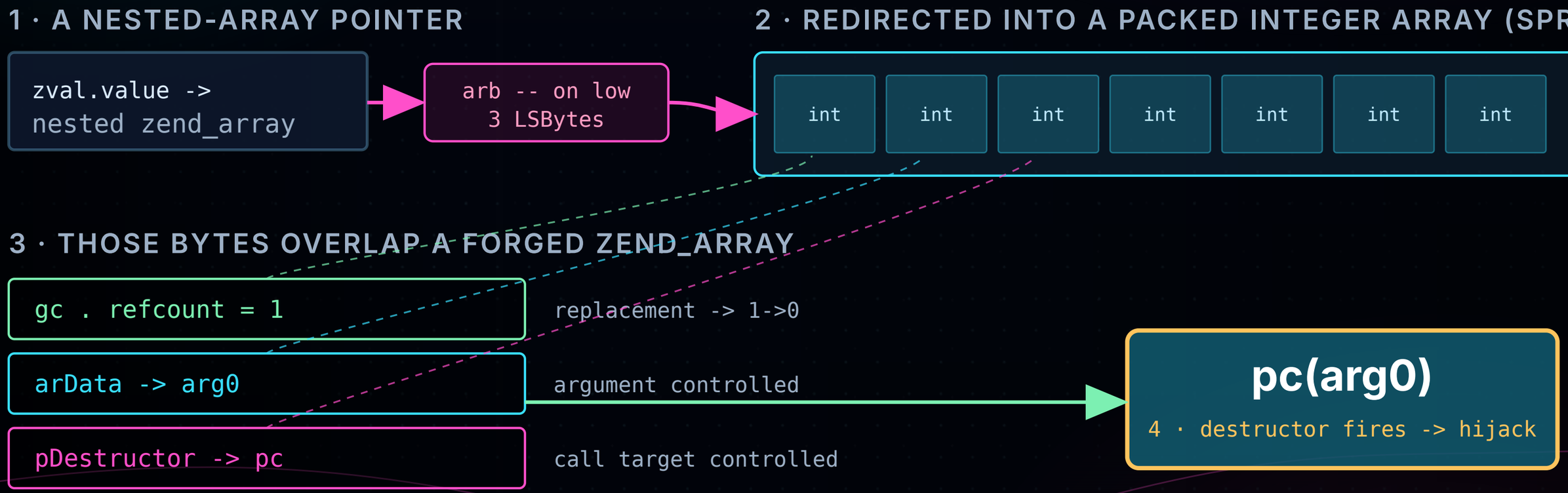
near each other

zend_heap	0x00007ffff4600000	rw-

PROBE PAGE-BY-PAGE FROM A KNOWN HEAP ANCHOR

The heaps sit near known code, zend_heap by libc, FPM_heap by php-fpm, so we probe from an anchor until a page faults (502).

Hijack via a destructor.



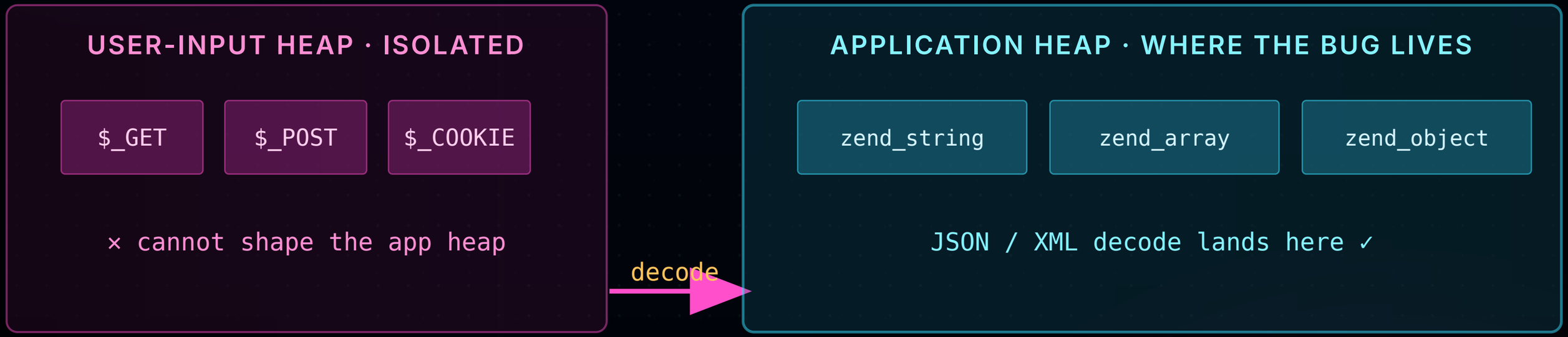
A forged array\'s refcount hits zero, PHP calls its destructor, and we control both the function and its argument.

IV

Doing it remotely

Everything so far needs a heap layout, built in one shot, with no raw bytes allowed.

Isolation has a gap.



Raw request fields are walled off, but the objects the app decodes from them land on the app heap, right where the bug is.

Duplicate keys = a heap script.

```
{
    "k": "A"*0x3000,
    "k": "B"*0x4000,
    "k": target,
    "k": null
}
```

Each value is allocated, all-but-one freed, an alloc/free script inside one request.

Integer arrays are a raw memory editor.

A packed, integer-keyed array stores a flat run of zvals.
Set each value → spray any 8-byte pattern.

V

End to end

One real CVE, one constrained byte, fully remote, from HTTP request to shell.

One real bug, fully remote.

CVE-2024-2961

glibc iconv OOB via a normal PHP app. In practice: one byte, value pinned to `0x48–0x4D`.

All four defenses on. No recon request.

WHERE THE WORK LIVES

The victim app is a few lines. The exploit is a heap program encoded entirely in one JSON body.

Build, place, trigger. Once.

build
spray fake bucket + zvals

place
page-level fengshui

forge
1-byte IF → fake zval

trigger
destructor → pc(arg0)

Leak, layout, and hijack all fold into a single HTTP body, there is no second request.

Quick demo.

VI

Does it hold up?

Real CVEs, all defenses on, measured for reliability, not a one-off crash.

We revived the dead CVEs.

TARGET	GOAL	OLD	OURS	REQS	SUCCESS
CVE-2024-2961	RCE	✗	✓	246	100%
CVE-2022-31626	RCE	✗	✓	285	100%
CVE-2019-6977	SBE	✗	✓	2	100%
CTF Case A ("I hate php" from SecurinetsCTF)	RCE	✗	✓	3	100%
CTF Case B ("php master" from N1CTF)	RCE	✗	✓	371	100%

Old public exploits: dead under hardening.
Rebuilt with IF + ZOP: revived, under 300 requests, 100% over 100 runs.

The roadmap, graded.

Already merged & effective: **Shadow Pointer** · **Unlink Abuse Prevention** , they forced attackers off freelist poisoning

PROTOTYPE

Read-only Metadata

Metadata falls, but by then the write is already strong, and the ASLR probe exposes writable structs anyway.

verdict: limited

IN DEVELOPMENT

Further Heap Isolation

Per-type dedicated heaps disrupt remote page-level shaping, the assumption our whole path rests on.

verdict: strong

PROPOSAL

Guard Pages

Fixed-size gaps do little here. A randomized variant injects real page-level entropy.

only if randomized

PROPOSAL

Freelist Randomization

Reorders slots in a freelist, but page-level spraying still walks past it.

verdict: partial

Isolation + randomization move the needle. Metadata protection helps least.

The smallest fix.

PATCH-HASHTABLE

Check the index stays within `num_used`. Kills Index Forgery at the source.

0.17% overhead

PATCH-REFCNT

Validate the target is an aligned `zend_refcounted` on the PHP heap. Kills arbitrary ++/--.

8.86% overhead

Good hardening works.

WHAT PHP GOT RIGHT

Good freelist hardening. Attackers are forced off the allocator.

WHAT'S LEFT

Built-in objects and decoder-driven layout.

If you want a common defense, harden all common paths.

- **Generic object hardening**
- **Object isolation**
- **Randomization**

Good hardening works.

If you want a common defense, harden all common paths.

Generic object hardening

Object isolation

Randomization

github.com/GhostFrankWu/PHP-security-research

THANK YOU

PHP's hardening moved the bar. We showed where it still needs to go, and shipped two fixes toward it.

Burning Tears of PHP's Memory Hardening

Yifan Wu · Xiaochuan Yu · Zhiyun Qian

Artifact + Docker: one-click reproduction

Black Hat USA 2026 Briefings